Prédiction des Prix de Maison

Machine Learning appliqué à la régression

Cahier de projet

Questions et consignes à réaliser soi-même

Comment utiliser ce document

Chaque partie contient des rappels théoriques courts, puis des questions numérotées
à traiter dans votre propre notebook. Aucune solution n'est fournie : c'est vous qui écrivez le code et justifiez vos choix.

Niveau : débutant → intermédiaire

Outils : Python, pandas, scikit-learn

Table des matières

1 Cadrage du projet **2**
1.1 Objectif . . . 2
1.2 Jeu de données . . . 2
1.3 Le pipeline à construire . . . 2
1.4 Livrable final du projet . . . 2

2 Partie 1 — Exploration des données (EDA) **3**

3 Partie 2 — Nettoyage des données **4**
3.1 Valeurs manquantes . . . 4
3.2 Valeurs aberrantes (outliers) . . . 4
3.3 Encodage des variables catégorielles . . . 5
3.4 Création de nouvelles variables (feature engineering) . . . 5
3.5 Colinéarité . . . 5

4 Partie 3 — Préparation pour la modélisation **6**

5 Partie 4 — Modélisation **7**

6 Partie 5 — Évaluation du modèle **8**
6.1 Métriques de régression . . . 8
6.2 Validation croisée . . . 8
6.3 Analyse des résidus . . . 8
6.4 Courbes d'apprentissage . . . 8
6.5 Importance des variables . . . 9
6.6 Pièges d'évaluation à vérifier vous-même . . . 9

7 Partie 6 — Pour aller plus loin (bonus) **10**

Grille d'auto-évaluation **11**

1 Cadrage du projet

1.1 Objectif

Construire, seul(e) et de bout en bout, un modèle qui prédit le prix de vente d'une maison à partir de ses caractéristiques. Vous devrez nettoyer un jeu de données réel, entraîner plusieurs modèles, et surtout être capable de justifier et d'évaluer rigoureusement vos résultats.

1.2 Jeu de données

Choisissez l'un des jeux suivants :

- **Ames Housing** (Kaggle, *House Prices - Advanced Regression Techniques*) : recommandé, très riche, beaucoup de valeurs manquantes et de variables catégorielles.
- **California Housing** (`sklearn.datasets.fetch_california_housing`) : plus simple, utile pour un premier essai rapide avant d'attaquer Ames Housing.

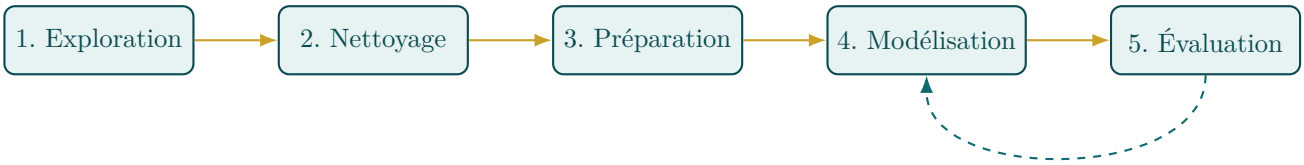
Question 1 — Point de départ

Chargez le jeu de données choisi dans un notebook. Vérifiez que le chargement est correct (dimensions, aperçu des premières lignes).

```
import pandas as pd

df = pd.read_csv("train.csv")    # ou fetch_california_housing()
# A vous de continuer...
```

1.3 Le pipeline à construire



1.4 Livrable final du projet

Livrable attendu

Un notebook unique (ou un dépôt structuré) contenant : l'exploration, le pipeline de nettoyage, l'entraînement d'au moins 3 modèles, leur évaluation comparée, et une conclusion écrite de quelques lignes qui justifie le modèle retenu.

2 Partie 1 — Exploration des données (EDA)

L'exploration guide toutes les décisions de nettoyage qui suivent : ne la sautez pas, même si elle semble moins "intéressante" que la modélisation.

Question 2

Combien de lignes et de colonnes contient le jeu de données ? Combien de variables sont numériques, combien sont catégorielles ? Y a-t-il une colonne identifiant qu'il faudra exclure du modèle ?

Question 3

Tracez la distribution de la variable cible (le prix). Est-elle symétrique ? Calculez sa *skewness*. Que suggérez-vous comme transformation, et pourquoi ?

Indice

Une skewness fortement positive suggère une transformation logarithmique de la cible ($\log(1+\text{prix})$). Cherchez la fonction `.skew()` de pandas et `np.log1p`.

Question 4

Identifiez les 10 variables numériques les plus corrélées avec la cible. Une forte corrélation garantit-elle une relation de cause à effet ?

Question 5

Choisissez 3 variables catégorielles et visualisez leur relation avec le prix (boxplot par catégorie). Lesquelles semblent avoir un pouvoir prédictif fort à l'œil nu ?

Livrable attendu

3 à 5 graphiques commentés (distribution de la cible, corrélations, boxplots), avec pour chacun une phrase qui explique ce qu'il révèle.

3 Partie 2 — Nettoyage des données

C'est la partie la plus déterminante du projet : un modèle sophistiqué ne compense pas des données mal préparées.

3.1 Valeurs manquantes

Question 6

Listez toutes les colonnes contenant des valeurs manquantes, avec leur pourcentage. Pour chacune, consultez la documentation du dataset : le NaN signifie-t-il une vraie absence de valeur (ex. pas de garage) ou une donnée réellement manquante ?

Attention

Ne traitez jamais toutes les colonnes de la même façon par automatisme. Une variable comme la qualité de piscine avec NaN quand il n'y a pas de piscine n'a pas le même traitement qu'une donnée de surface manquante par erreur de saisie.

Question 7

Pour chaque colonne concernée, choisissez et justifiez une stratégie d'imputation (suppression de la colonne, suppression des lignes, imputation par moyenne/médiane/mode, catégorie "Aucun", imputation par KNN). Implémentez-la.

Indice

Regardez du côté de `SimpleImputer` et `KNNImputer` de `sklearn.impute`. Une colonne avec plus de 40–50% de valeurs manquantes et peu de valeur prédictive est souvent supprimée plutôt qu'imputée.

Question 8

Vérifiez qu'il ne reste plus aucune valeur manquante dans votre jeu de données après traitement.

3.2 Valeurs aberrantes (outliers)

Question 9

Sur au moins 3 variables numériques clés, détectez les valeurs aberrantes à l'aide de la méthode de l'IQR (écart interquartile). Combien de lignes sont concernées pour chacune ?

Indice

Rappel : une observation est aberrante si elle sort de l'intervalle $[Q1 - 1.5\,IQR,\ Q3 + 1.5\,IQR]$. Complétez par une visualisation (nuage de points ou boxplot) pour confirmer visuellement.

Question 10

Pour chaque groupe d'outliers détecté, décidez : s'agit-il d'une erreur de saisie à supprimer, d'une vente atypique légitime à retirer de l'entraînement, ou d'une valeur correcte à conserver (éventuellement après transformation) ? Justifiez chaque décision par écrit.

3.3 Encodage des variables catégorielles

Question 11

Classez vos variables catégorielles en deux groupes : celles qui ont un ordre naturel (ex. qualité : mauvais → excellent) et celles qui n'en ont pas (ex. type de quartier). Quel encodage utiliserez-vous pour chaque groupe, et pourquoi ce choix plutôt qu'un autre ?

Question 12

Implémentez l'encodage choisi. Combien de colonnes obtenez-vous au total après un encodage One-Hot complet ? Est-ce potentiellement un problème (dimensionnalité, temps de calcul, risque de surapprentissage) ?

3.4 Création de nouvelles variables (feature engineering)

Question 13

Proposez au moins 3 nouvelles variables construites à partir des colonnes existantes (par exemple : âge du bien au moment de la vente, surface totale combinée, un ratio pertinent). Pour chacune, justifiez en une phrase pourquoi elle pourrait aider le modèle.

Question 14

Vérifiez si vos nouvelles variables sont plus corrélées à la cible que les variables d'origine dont elles sont dérivées. Que concluez-vous ?

3.5 Colinéarité

Question 15

Calculez le facteur d'inflation de la variance (VIF) de vos principales variables numériques. Quelles variables dépassent un seuil de 5 ? Que décidez-vous de faire de ces variables (les garder, en supprimer une, changer de modèle) ?

Livrable attendu

Un pipeline de nettoyage reproductible (idéalement un `ColumnTransformer`/`Pipeline` scikit-learn plutôt que des transformations manuelles non traçables), accompagné d'un court paragraphe qui justifie chaque décision prise (imputation, outliers, encodage, nouvelles variables, colinéarité).

4 Partie 3 — Préparation pour la modélisation

Question 16

Séparez vos données en un jeu d'entraînement et un jeu de test. Quelle proportion choisissez-vous et pourquoi ? À quel moment exact de votre pipeline ce découpage doit-il intervenir ?

Attention

Toute transformation apprise sur les données (moyenne pour l'imputation, paramètres du scaler, catégories du target encoding...) doit être ajustée uniquement sur le jeu d'entraînement, jamais sur l'ensemble complet. Vérifiez que votre pipeline respecte bien cette règle.

Question 17

Vos variables ont-elles besoin d'être mises à l'échelle (standardisation, normalisation) ? La réponse dépend-elle des modèles que vous envisagez d'entraîner en partie 4 ? Justifiez pour chaque modèle prévu.

5 Partie 4 — Modélisation

Question 18

Entraînez un modèle de référence (baseline) simple, par exemple une régression linéaire. Quel est son score ? Pourquoi ce modèle reste utile même s'il n'est pas le plus performant ?

Question 19

Choisissez et entraînez au moins 3 modèles de complexité croissante (par exemple : un modèle linéaire régularisé, une forêt aléatoire, un gradient boosting). Listez-les avec une phrase sur ce qui les différencie.

Question 20

Pour chacun des modèles choisis, identifiez un hyperparamètre clé et expliquez l'effet attendu de son augmentation ou de sa diminution sur le compromis biais-variance.

Livrable attendu

Un tableau récapitulatif : modèle, hyperparamètres utilisés, temps d'entraînement approximatif.

6 Partie 5 — Évaluation du modèle

C'est l'étape qui détermine si vos résultats sont dignes de confiance. Un modèle mal évalué peut sembler excellent en apparence et échouer complètement en conditions réelles.

6.1 Métriques de régression

Question 21

Calculez au moins le MAE, le RMSE et le R^2 pour chacun de vos modèles. Construisez un tableau comparatif.

Indice

RMSE $= \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$, et $R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$. Cherchez du côté de `sklearn.metrics`.

Question 22

Si vous avez transformé la cible en logarithme, à quel moment exact devez-vous "dé-transformer" vos prédictions pour interpréter l'erreur dans l'unité réelle (dirhams ou dollars) ? Faites-le et comparez avec l'erreur calculée sur l'échelle logarithmique.

6.2 Validation croisée

Question 23

Refaites l'évaluation de vos modèles avec une validation croisée en K-Fold (choisissez et justifiez la valeur de K). Le classement des modèles change-t-il par rapport à un split unique ? Que vous apprend l'écart-type des scores obtenus sur les différents folds ?

6.3 Analyse des résidus

Question 24

Pour votre meilleur modèle, tracez les résidus (erreur = valeur réelle − prédiction) en fonction des prédictions. Observez-vous un motif particulier (entonnoir, courbe) ? Qu'est-ce que cela révèlerait sur les limites du modèle ?

Question 25

Tracez un QQ-plot des résidus. Semblent-ils suivre approximativement une distribution normale ?

6.4 Courbes d'apprentissage

Question 26

Tracez les courbes d'apprentissage (erreur d'entraînement et de validation en fonction de la taille du jeu d'entraînement) pour votre meilleur modèle. Diagnostiquez : sous-apprentissage, surapprentissage, ou bon équilibre ? Quelle action concrète en découle ?

6.5 Importance des variables

Question 27

Calculez l'importance des variables pour votre meilleur modèle (la *permutation importance* est recommandée, plus fiable que l'importance native des arbres). Les 5 variables les plus importantes correspondent-elles à votre intuition issue de la partie 1 ?

6.6 Pièges d'évaluation à vérifier vous-même

Question 28

Relisez votre pipeline complet du début à la fin. Identifiez un endroit où une fuite de données (*data leakage*) aurait pu se produire si vous n'aviez pas fait attention. Expliquez comment vous l'avez évitée.

Question 29

Avez-vous ajusté des hyperparamètres en observant le score sur le jeu de test plus d'une fois ? Si oui, comment corrigeriez-vous votre méthodologie pour la suite ?

Livrable attendu

Un tableau comparatif final (modèle, MAE, RMSE, R^2, score CV moyen $\pm$ écart-type), les graphiques de résidus et de courbes d'apprentissage pour le meilleur modèle, et un paragraphe de conclusion qui justifie le modèle retenu.

7 Partie 6 — Pour aller plus loin (bonus)

Question 30

Optimisez les hyperparamètres de votre meilleur modèle avec une recherche systématique (`GridSearchCV` ou `RandomizedSearchCV`). Le gain de performance obtenu justifie-t-il la complexité et le temps de calcul ajoutés ?

Question 31

Essayez de combiner plusieurs modèles (par exemple un `VotingRegressor` ou un empilement/*stacking*). Observez-vous une amélioration par rapport à votre meilleur modèle seul ?

Question 32 — optionnelle

Sauvegardez votre modèle final sur disque. Décrivez comment vous procéderiez pour l'utiliser afin de prédire le prix d'une nouvelle maison qui n'a jamais été vue par le modèle.

Grille d'auto-évaluation

Avant de considérer le projet terminé, posez-vous ces questions :

Nettoyage des données

- ☐ Ai-je vérifié le type de chaque colonne avant de commencer ?
- ☐ Ai-je distingué les NaN qui signifient une absence réelle de ceux qui sont de vraies données manquantes ?
- ☐ Ma stratégie d'imputation est-elle justifiée pour chaque colonne, pas juste appliquée par automatisme ?
- ☐ Ai-je détecté et pris une décision réfléchie sur les outliers ?
- ☐ L'encodage catégoriel respecte-t-il la présence ou l'absence d'ordre entre catégories ?
- ☐ Ai-je vérifié la colinéarité de mes variables numériques ?
- ☐ Tout mon nettoyage est-il encapsulé dans un pipeline reproductible ?

Évaluation du modèle

- ☐ Le split train/test a-t-il eu lieu avant tout ajustement de transformateur ?
- ☐ Ai-je calculé plusieurs métriques, pas une seule isolément ?
- ☐ Ai-je utilisé une validation croisée plutôt qu'un seul split ?
- ☐ Ai-je analysé visuellement les résidus ?
- ☐ Ai-je tracé et interprété des courbes d'apprentissage ?
- ☐ Ai-je vérifié l'absence de fuite de données dans tout le pipeline ?
- ☐ Ai-je utilisé le jeu de test une seule fois, à la toute fin ?

Pour résumer

Si vous pouvez répondre "oui, et je peux le justifier" à chaque case, votre projet est solide sur le plan méthodologique — indépendamment du score final obtenu. Un score moyen mais honnêtement évalué vaut mieux qu'un score excellent obtenu par une fuite de données non détectée.